

Table of Contents

1. Executive Summary	2
Key Findings Summary	2
2. Project Root Analysis	3
Root Artifacts Requiring Cleanup	3
3. Source Code Structure	4
4. Domain Architecture	5
Domain Dependency Graph	5
Domain Issues	5
5. Components Architecture	6
6. Services, Providers, and State Management	7
7. Routing Architecture	8
8. Shared Infrastructure Layer	9
9. Backend and Server Architecture	10
10. Supabase Integration	11
11. Build System and Dependencies	12
12. CI/CD and Development Tooling	13
Immediate Refactor Tasks	13
13. Dependency Graph Analysis	14
14. Final Assessment	15
Architecture Score Breakdown	15
Overall Architecture Score	15
Top Strengths	15
Top Weaknesses	16

1. Executive Summary

VIXOR is a Solana-focused cryptocurrency trading terminal built as a full-stack React application. The architecture employs a domain-driven design (DDD) approach with 23 distinct domain modules, a shared infrastructure layer, and a TanStack Start framework providing SSR and server functions. The application integrates multiple external APIs for market data, AI-powered analysis through Vercel AI SDK with multiple LLM providers, and Web3 wallet connectivity for both EVM and Solana chains. The backend is deployed on Vercel serverless functions with Upstash Redis for rate limiting and caching.

The overall architecture demonstrates a mature understanding of domain separation and clean module boundaries. The dependency graph between domains forms a clean directed acyclic graph (DAG) with no circular dependencies. However, several significant issues were identified across the codebase: a non-existent CI/CD pipeline despite production deployment, a broken shadcn/ui configuration path, 3 duplicated hook modules, an unconfigured Storybook installation, missing test infrastructure, and several unused or underutilized dependencies. The application scores 5.8 out of 10 overall, reflecting strong domain architecture undermined by infrastructure gaps and missing development tooling.

6.2

Architecture

7.5

Strengths

4.1

Weaknesses

5.0

Tech Debt

5.5

Scalability

5.3

Maintainability

3.8

Enterprise Ready

Key Findings Summary

- **23 P0/P1 issues** identified across architecture, infrastructure, and configuration layers
- **Zero CI/CD** pipeline exists despite production deployment on Vercel
- **Zero test coverage** -- vitest is configured but no test runner script exists
- **Strong domain architecture** -- clean DAG, no circular dependencies, well-separated concerns
- **455 source files** remaining after Phase 1 cleanup (34 dead files removed)
- **39 authenticated routes** -- potential over-scoping for MVP stage
- **57 production dependencies** -- several potentially unused (pagedjs, oakscriptjs, sentiment)

2. Project Root Analysis

The project root contains the expected configuration files for a modern TypeScript/Vite project, along with several artifacts that indicate incomplete housekeeping. The presence of multiple lock files (pnpm-lock.yaml, bun.lock, package-lock.json) suggests the project has been used across different package managers without cleanup. Three accidentally created files exist in the root: **--timeout** (CLI typo artifact), **README (2).md**, and **components (2).json** (duplicates of existing files). Three SQL migration files also sit in the root directory instead of the supabase/migrations/ folder where they belong.

The vercel.json configuration is well-structured, specifying Node.js 22.x runtime, build command with 4GB memory allocation, and two cron jobs for daily signal generation (midnight UTC) and alert checking (00:30 UTC). The pnpm-workspace.yaml exists but the project does not appear to use workspace features, suggesting it was scaffolded from a monorepo template without removal. Multiple phase report markdown files (PHASE_A_REPORT.md, PHASE_B_REPORT.md, etc.) in the root suggest ongoing audit work that should be consolidated into a dedicated documentation directory.

Root Artifacts Requiring Cleanup

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
ARCH-001	P2	Multiple lock files	Confusion about package manager	Low	pnpm-lock.yaml, bun.lock, package-lock.json	Scaffolded from template, never cleaned
ARCH-002	P2	Accidental root files	Unprofessional repo appearance	Low	--timeout, README (2).md, components (2).json	CLI typos and copy-paste errors
ARCH-003	P2	Misplaced SQL files	Schema confusion	Low	3x root-level .sql files	Migration files moved incorrectly
ARCH-004	P2	Root-level reports	Cluttered repository root	Low	PHASE_*.md, DATA_LAYER_REPORT.md, VIXOR_UI_AUDIT.md	Audit artifacts not organized

Section Score: 4/10 -- Functional but messy root directory

3. Source Code Structure

The `src/` directory follows a well-organized structure with clear separation between routes, domains, shared infrastructure, and components. The top-level entry points (`start.ts`, `server.ts`, `router.tsx`) correctly bootstrap the TanStack Start application with SSR support, middleware configuration, and error handling. The `styles.css` file contains the complete VIXOR Design System V5 implemented using Tailwind CSS v4 with CSS-first configuration (no separate `tailwind.config.js`), defining theme tokens for colors, radius, spacing, motion, and typography including Inter for UI, JetBrains Mono for financial data, and Amiri for Arabic text support.

The `src/lib/` directory no longer exists after Phase 1 cleanup, which removed 34 dead re-export shims. However, the `shadcn/ui` `components.json` still references `@/lib/` as the utility path, which now resolves incorrectly. All utilities have been migrated to `src/shared/`, but the configuration has not been updated to reflect this. This is a P1 issue because it creates confusion for developers adding new `shadcn` components, who would expect to find `utils` at the aliased path.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
ARCH-005	P1	Broken <code>shadcn/ui</code> alias	New component generation fails	Medium	<code>components.json</code>	<code>utils.ts</code> deleted, alias not updated
ARCH-006	P1	Duplicated hooks (3)	Maintenance confusion	Medium	<code>src/hooks/</code> vs <code>src/shared/hooks/</code>	Incomplete migration
ARCH-007	P2	<code>src/integrations/</code> mirrors <code>shared/</code>	Dual source of truth for Supabase	Medium	<code>src/integrations/supabase/</code>	Legacy directory not cleaned up

Section Score: 7/10 -- Well-organized with minor migration leftovers

4. Domain Architecture

The domain layer is the strongest aspect of the VIXOR architecture. The project implements 23 domain modules following domain-driven design principles. Each domain encapsulates its business logic, types, and server functions independently. The dependency graph forms a clean directed acyclic graph (DAG) with the market domain serving as the foundational leaf node, flowing up through analysis, strategy, and trading to the copilot domain at the top. This hierarchy correctly reflects the business logic dependencies.

The independent domains (discovery, arbitrage, wallet, daily-loop, notes, signal-tracking, trades, user, watchlist, moxi, broker) have zero cross-domain dependencies, demonstrating excellent cohesion. The AI-related domains (copilot, debate, moxi) form a natural cluster with the trading domain serving as the integration point between AI intelligence and market operations. The analysis domain (22 files) is the largest and most complex, containing sub-engines for indicators, pattern detection, SMC concepts, regime detection, and risk-reward calculations.

Domain Dependency Graph

The dependency graph is a clean DAG from market (leaf) through backtest, analysis, and chart-intelligence, converging at the trading domain, with copilot at the apex. No circular dependencies exist. This is a strong indicator of mature architectural decision-making and disciplined module boundaries.

Domain Issues

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
ARCH-008	P1	discover/ has 2 separate domains	Confusion about which to use	Medium	domains/discover/ (2 dirs)	Scaffolding artifact, consolidated
ARCH-009	P2	copilot/ has 2 agent generations	Code bloat, unclear which is active	Low	copilot/agents/ vs copilot/ai-v4/	Legacy + new code coexist
ARCH-010	P1	No domain-level tests	Regressions undetected	High	All domains except arbitrage	Tests never written
ARCH-011	P2	experiment/ depends on LLM for mutations	Non-deterministic backtests	Medium	domains/experiment/	Architectural coupling to AI availability

Section Score: 8/10 -- Excellent DAG, minor consolidation needed

5. Components Architecture

The component layer is split into two clear categories: vixor/ (38 application-specific components) and ui/ (42 shadcn/ui primitives). The shadcn/ui components follow the standard new-york style with lucide icons, providing a consistent and accessible base layer. The application components include 5 TradingView chart variants (full, mini, technical analysis, ticker tape, and DEX-specific), which represent a significant investment in charting functionality. The MoxiAvatar, CoachOverlay, HunterScoreCard, GovernorRiskPanel, and AnalystReportPanel components implement the multi-agent AI interface.

The AgentResponseLayout component was recently introduced to deduplicate AI response rendering across agents. Storybook stories exist for 8 components (EmptyState, LiveDot, MiniSparkline, PaginationBar, ExpandableWidget, PageLayout, RouteLoading, atoms), demonstrating intent for component documentation. However, the Storybook infrastructure itself is not configured -- the .storybook/ directory is missing despite 5 Storybook packages being installed as dev dependencies. This means the 8 story files are currently dead code.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
ARCH-012	P1	Storybook installed but not configured	8 story files are dead code	Medium	.storybook/ (missing)	Stories written before infra setup
ARCH-013	P1	5 TradingView chart variants	Maintenance burden, potential duplication	Medium	TradingView*.tsx (5 files)	Incremental addition without refactoring
ARCH-014	P2	No component catalog/documentation	Onboarding friction for new devs	Low	All components	No Storybook or Doc running

Section Score: 6/10 -- Good base, chart variants need consolidation

6. Services, Providers, and State Management

The application employs a pragmatic state management approach combining TanStack React Query for server state, React Context for global UI state, and local useState/useReducer for component-level state. React Query is configured with conservative defaults (staleTime: 30s, refetchOnWindowFocus: false, retry: 1) that prevent infinite re-render loops, which was a known issue (React #310) addressed by the use-render-guard hook. Zustand is installed (v5.0.14) but used in only one file (virtual-list.tsx), making it an unnecessary dependency.

Four custom React Contexts are active: QueryClientProvider (TanStack), I18nProvider (EN/AR with RTL support), WalletProvider (Web3 wallet management), and GlobalErrorBoundary (inline class component). The I18nProvider supports English and Arabic with lazy-loaded translations and localStorage persistence for language preference. The WalletProvider handles MetaMask, Phantom, WalletConnect, and Telegram wallet connections with challenge-response authentication using nonce storage in Redis. The event system (VixorEvents) provides a typed in-process event bus with 20+ event types and optional persistence to Supabase.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
ARCH-015	P2	Zustand installed but barely used	Unnecessary dependency (48KB)	Low	package.json, virtual-list.tsx	Added for one use case, not adopted broadly
ARCH-016	P1	No global error monitoring in UI	Silent failures in production	High	Root layout, all pages	Sentry init exists but error boundary coverage
ARCH-017	P2	Event bus persistence is optional	Event data loss when disabled	Medium	shared/events/	Feature flag controlled, not default

Section Score: 6/10 -- Pragmatic but missing monitoring and error coverage

7. Routing Architecture

The routing system uses TanStack Router with file-based routing (v1.170.15), providing type-safe route definitions and automatic code generation via `routeTree.gen.ts`. The application defines 39 authenticated routes under the `_authenticated/` layout group, which is protected by a `beforeLoad` guard that validates the Supabase session and falls back to Telegram auto-signin. The auth route (`/auth`) uses SSR disabled (`ssr: false`) to support Telegram WebApp SDK initialization, which requires client-side-only rendering.

The route structure follows a flat pattern with pathless layout components (`-component.tsx` files) for complex pages like `copilot`, `daily-loop`, `swap`, and `token detail`. The root layout (`__root.tsx`) chains providers in the correct order: `QueryClientProvider`, `I18nProvider`, `WalletProvider`, `GlobalErrorBoundary`, `AppShell`, and `Outlet`. This provider chain ensures that all authenticated pages have access to server state, internationalization, wallet functionality, and error handling. However, 39 routes for a product still in MVP stage suggests potential over-scoping -- many of these pages (`yield`, `curves`, `arbitrage`, `backtest`, `experiments`, `perpetuals`, `predictions`, `communities`) appear to be aspirational features that may not have complete implementations behind them.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
ARCH-018	P1	39 routes for MVP-stage product	Diluted development focus	High	<code>src/routes/_authenticated/</code>	Feature creep -- too many pages too early
ARCH-019	P2	No route-level code splitting config	Large initial bundle	Medium	<code>vite.config.ts</code> , all routes	TanStack Start default may not be optimal
ARCH-020	P2	No 404/redirect for invalid routes	Poor UX for deep links	Low	<code>router.tsx</code>	No catch-all route defined

Section Score: 5/10 -- Too many routes, potential feature creep

8. Shared Infrastructure Layer

The shared layer (`src/shared/`) is the cross-cutting infrastructure backbone of the application. It contains 60+ files organized into clear modules: Supabase clients (browser + server), resilience patterns (Redis rate limiter, circuit breaker, LRU cache), LLM routing (multi-provider support for OpenAI, Anthropic, Groq, ZAI), event system, tool registry, memory store, notification channels, market data clients, i18n, hooks, utilities, and more. This layer demonstrates excellent infrastructure thinking with proper separation of concerns.

The resilience module is particularly well-designed, implementing a Redis-backed sliding window rate limiter with in-memory fallback, a circuit breaker pattern, and an LRU cache -- all critical for a production trading application that interfaces with multiple external APIs. The LLM router supports 4 providers (OpenAI, Anthropic, Groq, ZAI) with automatic provider detection and fallback. The tool registry implements a plugin architecture where tools can be registered, discovered, and executed with permission checking, enabling the AI copilot to safely interact with trading functions. The market data module provides multi-source price resolution with WebSocket clients for real-time data from Binance and DexScreener.

However, several issues exist in the shared layer. The Supabase integration is duplicated between `src/shared/supabase/` and `src/integrations/supabase/` (legacy directory). The credential vault and crypto utilities exist but their usage across the codebase should be audited. The notification system supports 4 channels (in-app, Telegram, email, webhook) but the email channel implementation needs verification. Three hooks are duplicated between `src/hooks/` and `src/shared/hooks/` (`use-mobile`, `use-render-guard`, `use-stable-server-fn`), indicating an incomplete migration.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
ARCH-021	P1	Supabase integration duplicated	Dual source of truth	Medium	<code>src/integrations/</code> vs <code>src/shared/supabase/</code>	Legacy directory not removed after migration
ARCH-022	P2	3 hooks duplicated across directories	Bug fixes must be applied twice	Medium	<code>use-mobile</code> , <code>use-render-guard</code> , <code>use-stable-server-fn</code>	Migration from hook to <code>shared/hooks/</code> incomplete
ARCH-023	P1	No shared error handling pattern	Inconsistent error UX across pages	High	All routes and domains	Each page handles errors differently

Section Score: 7/10 -- Strong infrastructure with duplication issues

9. Backend and Server Architecture

The backend uses Nitro (v3.0.260603-beta) as the server engine, deployed on Vercel serverless functions with Node.js 22.x runtime. API endpoints are defined in `server/api/` using `h3` `defineEventHandler` with a consistent pattern: `withRateLimit` wrapper, `handlePreflight` for CORS, and either `authenticateRequest` (JWT) or admin key validation for authorization. The security layer (`server/api/_security.ts`) implements proper CORS whitelisting, JWT authentication via Supabase `getUser()`, and admin authentication via `X-Admin-Key` header.

The server exposes 15 API endpoints covering health checks, metrics, cron jobs (signal generation, alert checking, reanalysis), admin operations (migrate, validate), AI streaming (copilot-stream), public market data (discover, sol-price, market-overview), webhook receivers (Telegram, Stars), and wallet management (connect, session). Rate limiting is consistently applied with Redis-backed sliding windows and proper `X-RateLimit` headers. The `copilot-stream` endpoint implements SSE (Server-Sent Events) for real-time AI response streaming with both global (120/min) and per-user (20/min) rate limits.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
ARCH-024	P0	No CI/CD pipeline	Deployments rely solely on Vercel auto-deploy	Critical	.github/workflows/ (missing)	Never set up GitHub Actions
ARCH-025	P0	No test runner script	Zero tests executed in CI or locally	Critical	package.json (no test script)	vitest configured but not wired
ARCH-026	P1	Nitro beta version	Potential instability in production	Medium	package.json (nitro 3.0.260603-beta)	Bleeding-edge version for features
ARCH-027	P1	No API versioning	Breaking changes affect all clients	Medium	All server/api/ endpoints	No /v1/ prefix or versioning strategy
ARCH-028	P2	No request validation schema	Malformed requests cause unhandled errors	Medium	All endpoints except wallet	Zod installed but not used for API validation

Section Score: 5/10 -- Good security, missing CI/CD and testing

10. Supabase Integration

Supabase serves as the primary database and authentication provider. The database schema comprises 32 tables covering trading operations (analyses, trades, daily_signals, price_alerts), AI features (copilot_conversations, copilot_messages, agent_audit_log, agent_jobs), user management (profiles, user_settings, user_memories, user_strategies, user_streaks), gamification (credit_points, points_balances, points_transactions, point_packs, premium_plans, premium_subscriptions, referrals), and system tables (notifications, payments, domain_events, vixor_decisions, experiment_generations, experiments). The auto-generated TypeScript types file is 1,654 lines, providing full type safety for database operations.

The dual-client pattern is well-implemented: a browser client with lazy singleton initialization and Proxy-based graceful degradation (getSupabaseOrNull vs getSupabaseOrThrow), and a server admin client using the service role key that bypasses Row Level Security. Auth is handled through a TanStack Start middleware (attachSupabaseAuth) that automatically injects the authenticated user context into server functions. The migration system includes 23 SQL migration files and a runtime migration checker accessible via the /api/migrate endpoint.

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
ARCH-029	P1	No RLS audit documentation	Security holes may exist unnoticed	High	supabase/migrations/	RLS policies not reviewed or documented
ARCH-030	P2	Admin client bypasses RLS entirely	Server-side must be trusted completely	Medium	shared/supabase/client.server.ts	By design but needs strict code review

Section Score: 7/10 -- Solid integration, needs RLS audit

11. Build System and Dependencies

The build system uses Vite (v7.3.1) with TanStack Start (v1.168.25) and Nitro (v3.0.260603-beta) as the full-stack framework. React 19.2.0 is the UI framework, TypeScript 5.8 provides type checking, and Tailwind CSS v4 handles styling with LightningCSS as the CSS transformer. The build pipeline includes a post-build script (fix-vercel-bundle.mjs) that patches the Vercel bundle for compatibility -- a sign that the framework combination has rough edges in production deployment.

The dependency footprint is large: 57 production packages and 18 dev dependencies. Several production dependencies appear to be unused or underutilized: pagedjs (PDF page layout, 0 imports found), oakscriptjs (TradingView scripting, minimal usage), sentiment (NLP analysis, not actively used in production code), and Zustand (state management, used once). The CCXT library (crypto exchange trading) is correctly externalized from the client bundle via Vite Rollup configuration. Security headers are properly configured in vite.config.ts including CSP, X-Content-Type-Options, Referrer-Policy, and Permissions-Policy, with the CSP whitelist covering all required external domains (TradingView, Telegram, Supabase, Binance, etc.).

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
ARCH-031	P1	Unused dependencies (4+)	Increased bundle size and attack surface	Medium	pagedjs, oakscriptjs, sentiment, zustand	Added for features that were not completed
ARCH-032	P1	Post-build bundle patching	Fragile deployment, breaks on framework updates	High	scripts/fix-vercel-bundle.mjs	Workaround for TanStack Start + Vercel incompatibility
ARCH-033	P2	No bundle size budget enforcement	Bundle can grow without warning	Medium	vite.config.ts	chunkSizeWarningLimit exists but no CI enforcement

Section Score: 5/10 -- Works but fragile, dependency bloat

12. CI/CD and Development Tooling

This is the weakest area of the VIXOR architecture. There is no CI/CD pipeline whatsoever. The `.github/workflows/` directory does not exist. All deployments rely entirely on Vercel auto-deploy from the main branch, meaning every push to main triggers a production deployment with zero automated quality gates. There are no automated tests, no lint checks, no type checking, no security scanning, and no performance benchmarks running on each deployment. This is a critical gap for a financial application handling real user data and trading operations.

Storybook is installed with 5 packages (`@chromatic-com/storybook`, `@storybook/addon-docs`, `@storybook/react-vite`, plus 2 more) but the `.storybook/` configuration directory is missing. Eight story files exist in `src/components/vixor/` but they are effectively dead code since Storybook cannot run without configuration. The vitest test framework is configured (`vitest.config.ts`) with Node environment, fork pool, and path aliases, but there is no test script in `package.json`, meaning tests cannot be run with a simple npm command. Only 3 test files exist in the entire codebase: `runner.test.ts` (`safe-exec`), and 2 test files in the `arbitrage` domain.

Immediate Refactor Tasks

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
ARCH-034	P0	Create GitHub Actions CI pipeline	Unblocks all quality gates	Critical	<code>.github/workflows/</code>	No CI was ever created
ARCH-035	P0	Add test script to <code>package.json</code>	Unblocks test execution	Critical	<code>package.json</code>	vitest configured but not wired
ARCH-036	P0	Configure Storybook	Unlocks 8 existing stories	High	<code>.storybook/</code>	Stories written before config
ARCH-037	P1	Update <code>components.json</code> utils path	Fixes <code>shadcn/ui</code> integration	Medium	<code>components.json</code>	<code>lib/</code> deleted in Phase 2
ARCH-038	P1	Remove unused dependencies	Reduces bundle and attack surface	Medium	<code>package.json</code>	<code>pagedjs</code> , <code>oakscriptjs</code> , <code>sentiment</code> , <code>zustand</code>

Section Score: 2/10 -- Critical infrastructure gap

13. Dependency Graph Analysis

The inter-module dependency graph was analyzed across all 23 domains and the shared layer. The graph is a clean directed acyclic graph (DAG) with no circular dependencies. The market domain is the most depended-upon module, serving as the foundation for analysis, backtest, chart-intelligence, and chart-truth. The trading domain sits at the convergence point, depending on market, analysis, and strategy. The copilot domain is at the apex, depending on trading, user, and watchlist. This hierarchy correctly mirrors the business logic flow: raw market data flows up through analysis and strategy, into trading operations, and finally into AI-assisted decision making.

The independent domains (12 of 23) demonstrate excellent separation of concerns. These domains have zero cross-domain dependencies and can be developed, tested, and deployed independently. The AI cluster (copilot, debate, moxi) forms a natural grouping around the AI intelligence layer. One area of concern is the experiment domain depending on LLM for strategy mutations -- this creates a non-deterministic dependency that could affect backtest reliability. The discover domain exists in two separate directories (domains/discover/ with 16 files and domains/discover/ simpler with 3 files), which is a consolidation opportunity.

Section Score: 8/10 -- Clean DAG, excellent modularity

14. Final Assessment

Architecture Score Breakdown

Area	Score	Notes
Project Root	4.0/10	Functional but cluttered with artifacts
Source Structure	7.0/10	Well-organized, minor migration gaps
Domain Architecture	8.0/10	Excellent DAG, no circular deps
Components	6.0/10	Good base, chart variants need work
Services/State	6.0/10	Pragmatic, missing monitoring
Routing	5.0/10	Too many routes for MVP stage
Shared Layer	7.0/10	Strong infra, duplication issues
Backend/Server	5.0/10	Good security, no CI/CD
Supabase	7.0/10	Solid integration, needs RLS audit
Build System	5.0/10	Works but fragile, dep bloat
CI/CD Tooling	2.0/10	Critical gap -- nothing automated
Dependency Graph	8.0/10	Clean DAG, excellent modularity

Overall Architecture Score

5.8 / 10

VIXOR demonstrates strong domain architecture with excellent module boundaries and a clean dependency graph. The shared infrastructure layer is well-designed with proper resilience patterns (rate limiting, circuit breaking, caching). However, the architecture is significantly undermined by the complete absence of CI/CD pipelines, test infrastructure, and development tooling (Storybook). The build system relies on a fragile post-build patching step, and the dependency footprint includes several unused packages. For a financial application handling real user data, these infrastructure gaps represent serious risks that should be addressed before scaling the user base.

Top Strengths

- Clean domain-driven architecture with 23 well-separated modules

- Zero circular dependencies in the domain graph
- Comprehensive security layer (JWT auth, rate limiting, CORS, CSP headers)
- Multi-provider LLM routing with fallback support
- Resilient shared infrastructure (Redis rate limiter, circuit breaker, LRU cache)
- Proper Supabase dual-client pattern (browser + admin)

Top Weaknesses

- Zero CI/CD pipeline -- no automated quality gates on deployment
- Zero test coverage -- vitest configured but not executable
- Storybook installed (5 packages) but never configured
- 39 routes for an MVP-stage product -- potential feature creep
- Post-build bundle patching for Vercel compatibility -- fragile
- Several unused production dependencies increasing bundle size